

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ДОПУЩЕН К ЗАЩИТЕ

Заведующий кафедрой

_____ / Пухова Е. А. /

Руководитель образовательной программы

_____ / Даньшина М. В. /

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по теме:

**ВЕБ-ПРИЛОЖЕНИЕ СЕГМЕНТИРОВАННОЙ РАССЫЛКИ
УВЕДОМЛЕНИЙ ДЛЯ ДОНОРОВ КРОВИ**

по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Студент: _____ / Иванов Иван Иванович, 221–322/
подпись *ФИО, группа*

Руководитель ВКР: _____ / Иванов Иван Иванович/
подпись *ФИО, уч. звание и степень*

Москва 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное
Образовательное учреждение высшего образования
МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
(МОСКОВСКИЙ ПОЛИТЕХ)

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль) «Веб-технологии»

Тема ВКР	
ПРАКТИЧЕСКИЙ РЕЗУЛЬТАТ	
Назначение	
Основные функции	1. Список функций
Используемые технологии и платформы	Перечисление технологий.

ВЫПОЛНЕНИЕ РАБОТЫ	
Решаемые задачи	1. Список задач
Состав технической документации	1. Список документации
Состав графической части	1. Список графической части

ПЛАН РАБОТЫ НАД ВКР

[illegible]

РУКОВОДИТЕЛЬ ОП:

«___»_____2026, _____ / Даньшина Марина Владимировна /
подпись *ФИО, уч. звание и степень*

РУКОВОДИТЕЛЬ ВКР:

«___»_____2026, _____ / Иванов Иван Иванович /
подпись *ФИО, уч. звание и степень*

СТУДЕНТ:

«___»_____2026, _____ / Иванов Иван Иванович, 221–322 /
подпись *ФИО, группа*

АННОТАЦИЯ

Выпускная квалификационная работа (далее ВКР) на тему: «Тема».

Цель ВКР – ...

Наполнение аннотации смотрите в методичке.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 ЗАГОЛОВОК 1	11
1.1 Заголовок 2	11
1.2 Вывод	12
2 РЕАЛИЗАЦИЯ	13
2.1 Заголовок	13
2.2 Вывод	13
3 ЗАГОЛОВОК	14
3.1 Заголовок	14
3.2 Вывод	15
ЗАКЛЮЧЕНИЕ	16
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17
ПРИЛОЖЕНИЕ А	18

ВВЕДЕНИЕ

Актуальность темы. В современной практике обеспечения качества сложных программных комплексов проведение нагрузочного тестирования требует создания стабильной и управляемой среды. Одной из ключевых проблем является необходимость имитации внешних систем, с которыми взаимодействует объект тестирования. Использование имитационных сервисов («заглушек») позволяет изолировать тестируемую систему, однако управление большим количеством таких сервисов, их развертывание на удаленных серверах и контроль их состояния в ручном режиме существенно замедляют процесс подготовки испытаний.

В условиях реализации политики импортозамещения в Российской Федерации актуальность приобретает разработка отечественных инструментов управления инфраструктурой, способных заменить зарубежные программные продукты и обеспечить полноценное функционирование стендов нагрузочного тестирования в среде защищенных операционных систем, таких как ОС Astra Linux. Таким образом, создание системы, автоматизирующей жизненный цикл имитационных сервисов и предоставляющей механизмы контроля трафика, является своевременной и практически значимой задачей.

Цель работы – проектирование и разработка автоматизированной информационной системы для централизованного управления распределенной инфраструктурой имитационных сервисов.

Задачи исследования:

1. Провести анализ предметной области и существующих подходов к управлению имитационными средами в процессах обеспечения качества ПО.
2. Спроектировать архитектуру программного комплекса, обеспечивающую гибкое масштабирование и работу в различных режимах.
3. Обосновать выбор технологического стека и программных средств реализации системы.

4. Разработать алгоритмы динамического проксирования трафика, механизмы контроля интенсивности запросов и программные интерфейсы для автоматизации управления.

5. Реализовать систему хранения конфигураций и контроля состояний исполняемых файлов.

6. Провести тестирование разработанного комплекса в среде ОС Astra Linux и оценить эффективность реализованных механизмов мониторинга и управления.

Объект исследования – инфраструктура запуска и эксплуатации имитационных сервисов (заглушек).

Предмет исследования – механизмы и алгоритмы автоматизированного управления и контроля состояния имитационных сервисов.

Научная новизна работы заключается в разработке алгоритма централизованного управления распределенной средой, поддерживающего индивидуальную настройку ограничений интенсивности запросов (Rate Limiting) для каждого сервиса, а также в реализации программного интерфейса (API) для автоматического контроля состояния загрузок непосредственно из скриптов нагрузочного тестирования.

Практическая значимость работы состоит в создании программного инструмента, позволяющего отказаться от использования зарубежных систем управления, упростить эксплуатацию инфраструктуры имитационных сервисов и повысить уровень автоматизации процесса нагрузочного тестирования.

Положения, выносимые на защиту:

1. Архитектура программного комплекса, поддерживающая гибкое масштабирование системы от локального запуска на одном хосте до формирования распределенного кластера.

2. Механизм динамического проксирования запросов, реализующий прозрачную маршрутизацию входящего трафика к целевым имитационным сервисам.

3. Алгоритм контроля интенсивности запросов (Rate Limiting), обеспечивающий стабильность работы имитационной среды при пиковых нагрузках.

4. Методика формирования и экспорта метрик производительности для непрерывного мониторинга состояния инфраструктуры загрузок.

Структура работы. Выпускная квалификационная работа включает введение, три главы основного текста, заключение, список использованных источников и приложения. Во введении обоснована актуальность исследования, поставлены цель и задачи, определены объект и предмет работы, указаны научная новизна и практическая значимость полученных результатов. В первой главе проводится анализ предметной области, исследуются существующие аналоги и методы управления имитационными средами. На основе сравнительного анализа обосновывается выбор технологий и формируются требования к системе. Во второй главе описывается проектирование архитектуры системы, структуры хранения данных и логики работы основных модулей. Приводятся схемы взаимодействия компонентов и алгоритмы реализации функций проксирования и управления кластером. В третьей главе рассматриваются вопросы практической реализации, настройки и эксплуатации системы в среде ОС Astra Linux. Приводятся результаты тестирования, подтверждающие корректность работы разработанных механизмов управления и мониторинга. В заключении сформулированы основные выводы, подтверждающие решение поставленных задач и достижение цели работы.

1 ЗАГОЛОВОК 1

1.1 Заголовок 2

Пример цитирования [1] или [2] или [3] или [4].

Пример создания таблиц 1, в которую были занесены основные аспекты каждой реализации.

Таблица 1 – Сравнительная таблица ролевых моделей

Модель	Подходит для CRM	Гибкость	Масштабируемость	Сложность
RBAC	Да	Средняя	Высокая	Низкая
ABAC	Да	Высокая	Высокая	Средняя
DAC	Нет	Высокая	Низкая	Средняя
MAC	Нет	Низкая	Средняя	Высокая

На основе проведенного анализа и сравнения различных моделей управления доступом можно выделить два наиболее подходящих подхода: RBAC и ABAC.

Пример таблицы с разрывом на две страницы 2.

Таблица 2 – Сравнение технологий для разработки клиентской части

Характеристика	Next.js	Nuxt.js	Angular
1	2	3	4
Базовая технология	React	Vue.js	Фреймворк от Google
Язык разработки	JavaScript / TypeScript	JavaScript / TypeScript	TypeScript

Продолжение таблицы 2

1	2	3	4
Архитектура	Компонентная	Компонентная	Модульная
Сложность	Умеренная	Умеренная	Высокая
Производительность	Высокая	Высокая	Средняя на больших проектах
Размер сообщества	Большой	Большой, но меньше Next.js	Большое
Скорость разработки	Высокая	Высокая	Средняя
Гибкость	Высокая	Высокая	Ограниченная
Поддержка TypeScript	Отличная	Хорошая	Нативная

1.2 Вывод

Выводы по главе

2 РЕАЛИЗАЦИЯ

2.1 Заголовок

Пример ссылки на главу в главе 3.

Пример кавычек «RegisterUser» и длинного тире.

2.2 Вывод

Выводы по главе.

3 ЗАГОЛОВОК

3.1 Заголовок

В рамках ВКР удалось реализовать следующие ключевые функции:

а) список

Пример изображения и ссылки на него 1.

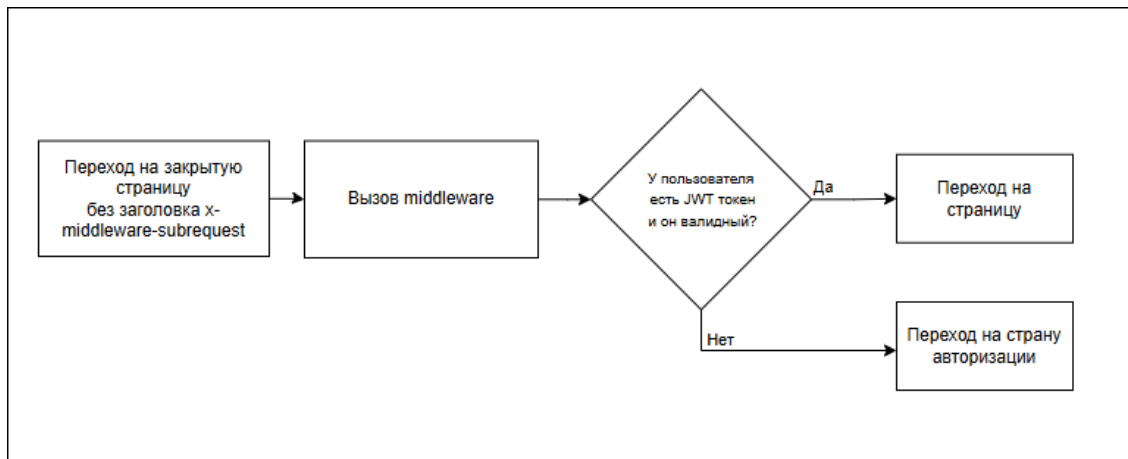


Рисунок 1 – Обработка запросов на переход без заголовка
Пример кода и ссылка на него 3.1.

Листинг 3.1 – Подключение CSRF-токена

```
1 import cookieParser from 'cookie-parser';
2 import csrf from 'csrf';
3 async function bootstrap() {
4   const app = await NestFactory.create(AppModule);
5   app.use(cookieParser());
6   app.use(
7     csrf({
8       cookie: {
9         httpOnly: true, // токен CSRF недоступен через JS
10        secure: true,
11        sameSite: 'strict',
12      },
13    }),
14  );
15  app.useGlobalFilters(new HttpExceptionFilter());
16  app.useGlobalPipes(new ValidationPipe({ whitelist: true }));
17  app.enableCors({origin: process.env.CORS, credentials: true});
18  await app.listen(process.env.PORT ?? 3000);
19 }
```

3.2 Вывод

Вывод по главе

ЗАКЛЮЧЕНИЕ

Описание проделанной работы

В результате работы были выполнены следующие задачи:

1. Задача 1
2. Задача 2

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Российская Федерация. Законы. О донорстве крови и её компонентов : Федеральный закон от 20.07.2012 №125-ФЗ. – Собрание законодательства Российской Федерации. 2012. № 30. Ст. 4176.
2. World Health Organization. World Health Statistics 2025: Monitoring Health for the SDGs / World Health Organization. – Geneva: World Health Organization, 2025. – 76 p.
3. Дронов, В. А. Laravel 9. Быстрая разработка веб-сайтов на PHP / В. А. Дронов. – Санкт-Петербург: БХВ-Петербург. – 768 с.
4. NestJS – A progressive Node.js framework. – 2025. – [Электронный ресурс] Дата обращения: 14.05.2025. <https://docs.nestjs.com>.

ПРИЛОЖЕНИЕ А

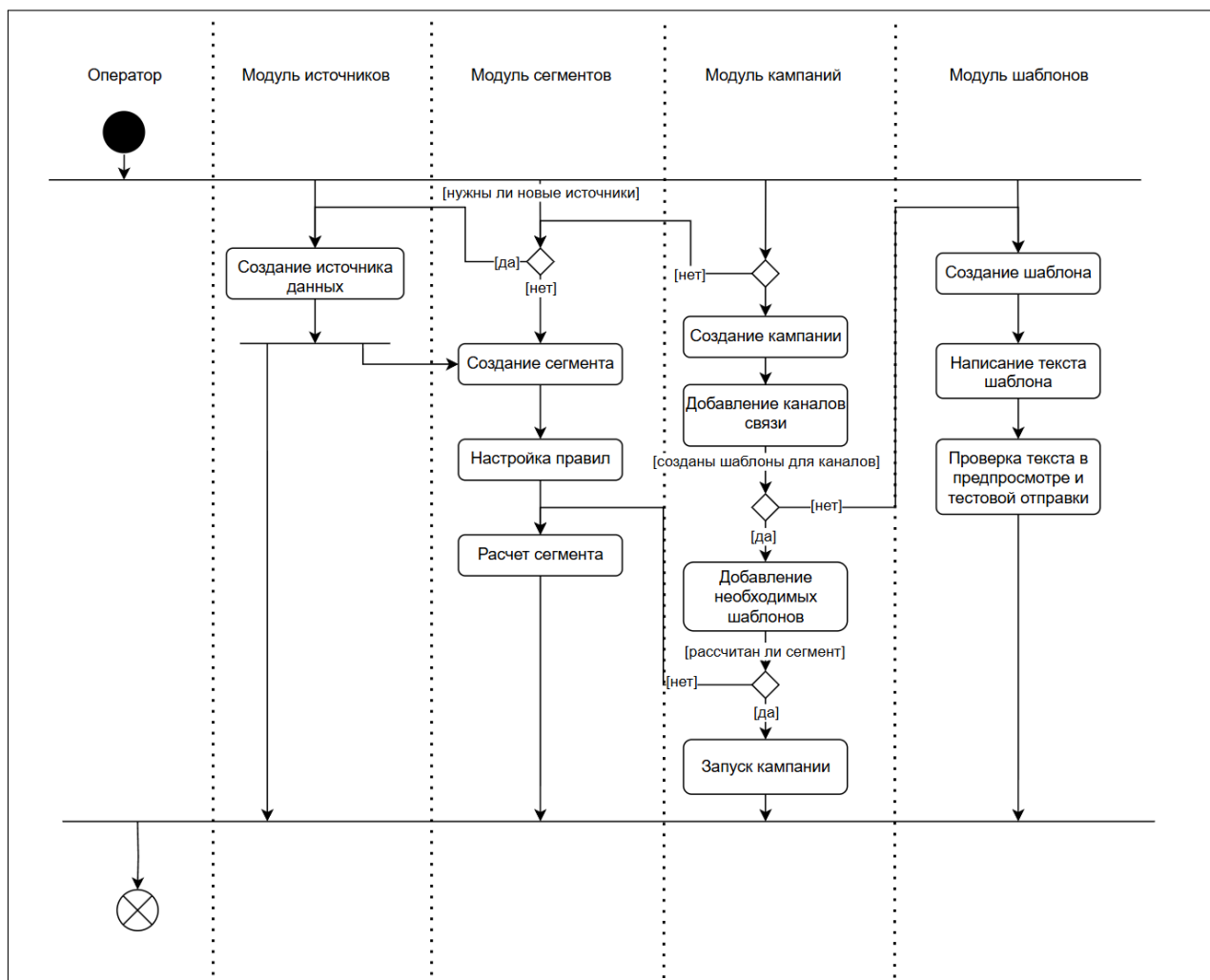


Рисунок А.1 – Создание кампании сегментированных рассылок

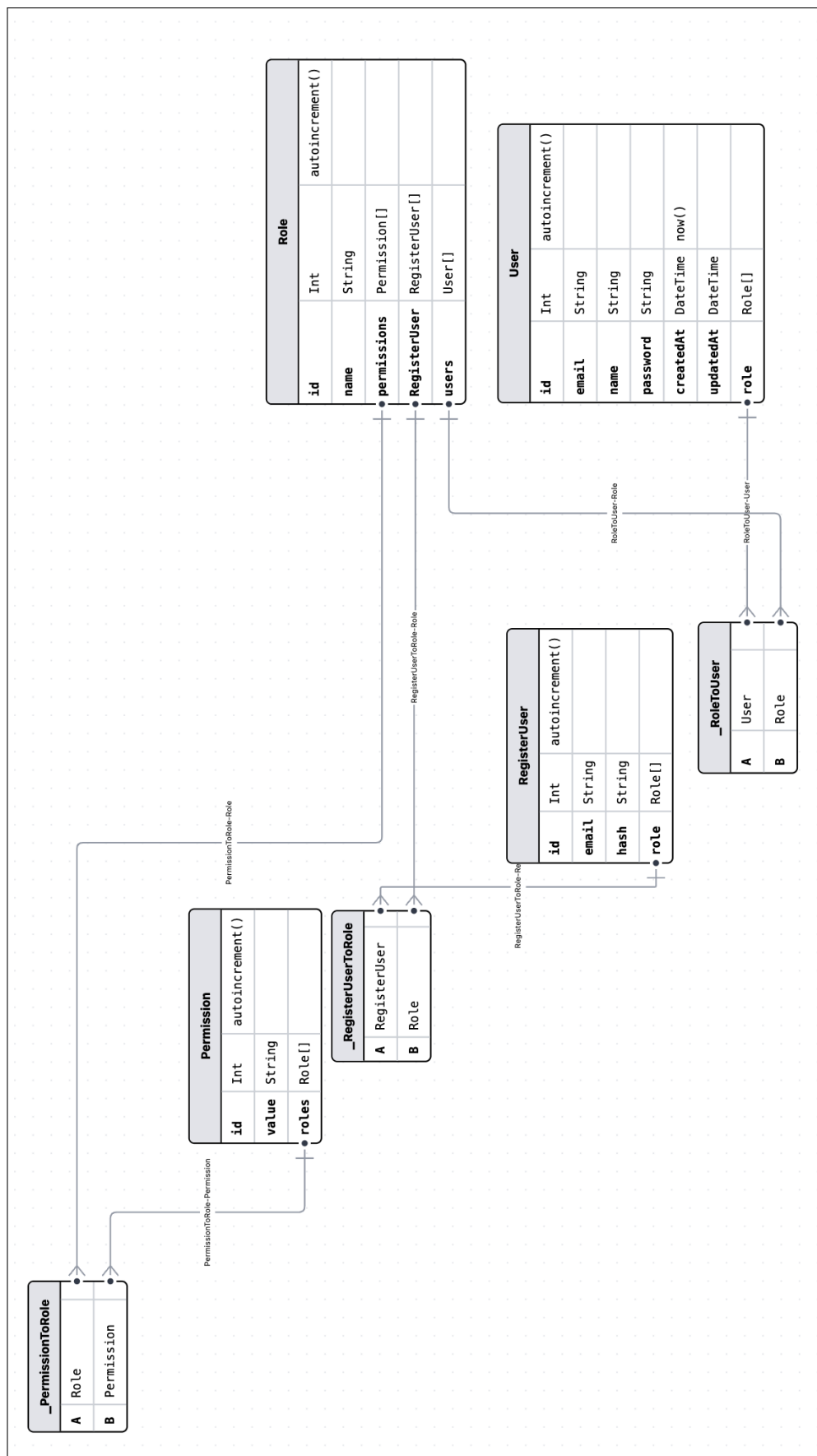


Рисунок А.2 – Даталогическая модель пользователя

Листинг 3.2 – Реализация «permissions.guard.ts»

```
1 @Injectable()
2 export class PermissionsGuard implements CanActivate {
3   constructor(
4     private reflector: Reflector,
5     private permissionsService: PermissionsService,
6   ) {}

7   canActivate(context: ExecutionContext): boolean {
8     try {
9       const requiredPermissions = this.reflector
10         .get<string[]>('permissions', context.getHandler())
11         .join('_');
12       if (!requiredPermissions) { return true }
13       const request = context.switchToHttp().getRequest();
14       const user = request.user;
15       return this.permissionsService.hasPermission(user,
16         ↪ requiredPermissions);
17     } catch {
18       return false;
19     }
20 }
```

Листинг 3.3 – Конфигурация для «docker-compose»

```
1 services:
2   documentation:
3     build: ./docs
4     ports:
5       - '3002:80'
6   app:
7     build: ./backend
8     ports:
9       - '3000:3000'
10    depends_on:
11      - db
12      - donorBb
13    env_file: ./backend/.env
14    environment:
15      - DATABASE_URL=postgresql://${DATABASE_USER}:${DATABASE_PASSWORD}@db:5432/${DATABASE_NAME}
16      - DONOR_DATABASE_URL=postgresql://${DONOR_DATABASE_USER}:${DONOR_DATABASE_PASSWORD}@donorBb:5432/${DONOR_DATABASE_NAME}
17      - DONOR_DATABASE_PORT=5432
18  frontend:
19    build: ./frontend
20    ports:
21      - '3001:3000'
22    depends_on:
23      - app
24    env_file: ./frontend/.env
25    environment:
26      - NEXT_PUBLIC_BASE_URL=/api
27  nginx:
28    image: nginx:latest
29    ports:
30      - '80:80'
31      - '443:443'
```

Продолжение листинга 3.3

```
1     volumes:
2         - ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
3         - /etc/letsencrypt/:/etc/letsencrypt:ro
4     depends_on:
5         - frontend
6         - app
7         - documentation
8     db:
9         image: postgres:16
10        ports:
11            - '5432:5432'
12        env_file:  ./backand/.env
13        environment:
14            - POSTGRES_USER=${DATABASE_USER}
15            - POSTGRES_PASSWORD=${DATABASE_PASSWORD}
16            - POSTGRES_DB=${DATABASE_NAME}
17        volumes:
18            - db_data:/var/lib/postgresql/data
19    donorBb:
20        image: postgres:16
21        ports:
22            - '5433:5432'
23        env_file:  ./backand/.env
24        environment:
25            - POSTGRES_USER=${DONOR_DATABASE_USER}
26            - POSTGRES_PASSWORD=${DONOR_DATABASE_PASSWORD}
27            - POSTGRES_DB=${DONOR_DATABASE_NAME}
28        volumes:
29            - donor_data:/var/lib/postgresql/data
30    volumes:
31        db_data:
32        donor_data:
```